

¿Qué Front-End elegimos? Análisis de React para el desarrollo de una aplicación web

1. Introducción

La elección de una tecnología Front-End condiciona aspectos importantes del desarrollo de una aplicación web, como la organización del código, la reutilización de componentes, la gestión del estado de la interfaz y las herramientas que necesita el equipo de desarrollo. Por este motivo, la decisión no debería basarse únicamente en la popularidad de una tecnología, sino en su adecuación a las características del proyecto y a los conocimientos del equipo.

Para este trabajo se ha seleccionado React, una tecnología orientada a la construcción de interfaces de usuario mediante componentes reutilizables. La documentación oficial define React como una biblioteca de JavaScript para renderizar interfaces de usuario y destaca la utilización de componentes como unidades fundamentales de construcción ([React, s. f.-a](#)).

El objetivo de este trabajo es analizar sus características, conocimientos necesarios, ventajas e inconvenientes y valorar su utilización en el caso propuesto de una academia con cursos, filtros, inscripción y área privada del alumnado.

2. Identidad de React

React es una biblioteca de JavaScript para la construcción de interfaces de usuario, no un framework completo. Su principal concepto de organización es el componente: una parte de la interfaz que puede encapsular su propia lógica y representación visual. Los componentes pueden combinarse y anidarse para construir interfaces de mayor complejidad ([React, s. f.-a](#)).

Esta característica permite dividir una aplicación en elementos independientes y reutilizables. Por ejemplo, en la aplicación de una academia podrían existir componentes como Curso, ListaCursos, FiltroCursos, FormularioInscripcion o PerfilAlumno. Esta división facilita que diferentes partes de la aplicación puedan desarrollarse y mantenerse de forma independiente. Una diferencia importante respecto a un framework como Angular es precisamente su alcance. Angular se define oficialmente como un framework web que proporciona un conjunto amplio de herramientas, APIs y bibliotecas para desarrollar aplicaciones ([Angular, s. f.](#)).

React, en cambio, se centra fundamentalmente en la construcción de la interfaz, por lo que otras necesidades de la aplicación pueden resolverse mediante herramientas adicionales.

React surgió originalmente en Facebook y actualmente su desarrollo se realiza mediante una combinación de un equipo dedicado y una amplia comunidad de colaboradores de código abierto. La documentación oficial señala que React cuenta con un equipo dedicado a tiempo completo y más de mil colaboradores de código abierto.

Además, React se distribuye bajo licencia MIT y su desarrollo se realiza de forma abierta mediante GitHub ([React, 2026](#)).

3. Conocimientos y herramientas necesarios

Para trabajar con React es necesario disponer principalmente de conocimientos de HTML, CSS y JavaScript. La propia documentación de React presenta los componentes como

funciones de JavaScript que devuelven marcado y explica el uso de JSX para expresar la interfaz dentro del código JavaScript ([React, s. f.-a](#)).

Los conocimientos considerados necesarios son:

- HTML, para comprender la estructura de los documentos y los elementos de la interfaz.
- CSS, para definir la presentación visual.
- JavaScript, puesto que React se basa en este lenguaje.
- Conceptos de programación como funciones, objetos, arrays y eventos.
- Conceptos propios de React, especialmente componentes, propiedades (props) y estado.

La documentación oficial explica que las aplicaciones React se construyen mediante componentes y que estos pueden recibir datos mediante props y mantener información que cambia durante la interacción mediante el estado ([React, s. f.-c](#)).

El desarrollo de una aplicación React suele apoyarse en herramientas adicionales de desarrollo, como Node.js, un gestor de paquetes y herramientas de construcción o build. Sin embargo, estas herramientas no deben confundirse con React propiamente dicho. React es la biblioteca de interfaz, mientras que el resto del entorno depende de la configuración y de las necesidades del proyecto.

Por tanto, puede distinguirse entre:

Obligatorio o fundamental:

- HTML.
- CSS.
- JavaScript.
- React y sus conceptos fundamentales.

Complementario:

- Herramientas de construcción.
- Gestor de paquetes.
- Librerías para necesidades concretas de la aplicación.
- Herramientas de pruebas, análisis de código o gestión del estado cuando el proyecto las requiera.

Esta separación es relevante porque una de las características de React es precisamente que no obliga a utilizar una solución única para todas las necesidades de una aplicación.

4. Ventajas de React

4.1. Reutilización mediante componentes

Una de las principales ventajas de React es su modelo basado en componentes. La interfaz puede dividirse en piezas independientes y reutilizables. Según la documentación oficial, los componentes pueden ser pequeños, como un botón, o representar partes mucho mayores de una página ([React, s. f.-a](#)).

Esta característica resulta especialmente útil en aplicaciones con muchos elementos repetidos. En el caso de la academia, un mismo componente Curso podría utilizarse para representar todos los cursos del catálogo. Si posteriormente se modifica la forma en que se muestra el

nombre, precio o duración de un curso, el cambio puede realizarse en el componente correspondiente.

4.2. Organización de interfaces complejas

React propone dividir una interfaz en una jerarquía de componentes y gestionar el flujo de datos entre ellos. Su documentación recomienda comenzar dividiendo visualmente la interfaz en componentes y posteriormente determinar qué información debe mantenerse como estado ([React, s. f.-c](#)).

Esta metodología puede facilitar el mantenimiento de aplicaciones que contienen muchas visitas e interacciones. En el caso de la academia, permite separar claramente el catálogo, los filtros, el formulario de inscripción y el área privada del alumnado.

4.3. Flexibilidad

Otra ventaja importante es que React se centra en la interfaz y permite seleccionar herramientas adicionales según las necesidades del proyecto. Esto puede resultar beneficioso cuando el equipo quiere adaptar la arquitectura a sus necesidades concretas.

La flexibilidad, sin embargo, debe entenderse como una ventaja condicionada. Una mayor libertad también implica que el equipo tiene que tomar más decisiones técnicas. Por tanto, esta característica es especialmente interesante cuando el equipo tiene suficiente experiencia para establecer unas convenciones claras.

5. Inconvenientes y limitaciones

5.1. Necesidad de elegir herramientas adicionales

Una consecuencia de que React sea una biblioteca y no un framework completo es que algunas necesidades habituales no quedan determinadas por React. El equipo puede tener que decidir qué soluciones utilizar para aspectos como el enrutamiento, determinadas estrategias de gestión del estado o la arquitectura general.

Esto proporciona flexibilidad, pero también aumenta la cantidad de decisiones que deben tomarse. En un equipo con poca experiencia puede producirse una arquitectura poco homogénea si cada desarrollador adopta soluciones diferentes.

5.2. Ecosistema amplio

El ecosistema de React ofrece una gran cantidad de herramientas y librerías. Esto puede resultar positivo porque existen soluciones para numerosas necesidades, pero también puede dificultar la selección de una alternativa adecuada.

El problema no consiste en que existan muchas herramientas, sino en que el equipo debe evaluar cuáles son necesarias y cuáles añaden complejidad innecesaria. Para una aplicación pequeña, incorporar numerosas dependencias puede ser contraproducente.

5.3. Curva de aprendizaje

Aunque los fundamentos de React pueden aprenderse progresivamente, desarrollar aplicaciones completas requiere comprender conceptos adicionales como componentes, estado, propiedades, flujo de datos y las herramientas utilizadas alrededor de React.

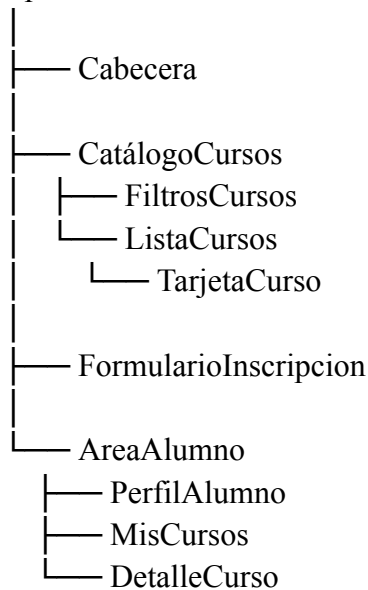
Por este motivo, React no debería considerarse simplemente una solución que elimina la necesidad de conocimientos de programación. La propia documentación explica que aprender React implica aprender a construir interfaces mediante componentes y utilizar JavaScript para expresar su comportamiento ([React, s. f.-a](#)).

6. Aplicación al caso de la academia

El proyecto planteado consiste en una aplicación web para una academia que dispone de un listado de cursos, filtros, un formulario de inscripción y un área del alumnado. Además, el servidor y la API ya existen.

React sería adecuado para organizar la interfaz mediante una estructura de componentes similar a la siguiente:

Aplicación Academia



Esta organización sigue el principio de dividir la interfaz en componentes y establecer relaciones entre ellos. React propone precisamente este proceso de descomposición de la interfaz como una de las primeras fases para construir una aplicación ([React, s. f.-c](#)).

Funciones que resolvería el navegador.

El Front-End podría encargarse de:

- Mostrar el catálogo de cursos.
- Aplicar filtros visuales al listado.
- Gestionar la interacción con botones y formularios.
- Mostrar mensajes de validación relacionados con el formato de los datos.
- Actualizar la interfaz cuando cambie el estado de la aplicación.
- Mostrar el área privada del alumnado una vez obtenida la información correspondiente de la API.

React permite que los cambios de estado provoquen la actualización de la interfaz, lo que resulta apropiado para elementos interactivos como filtros y formularios ([React, s. f.-c](#)).

Funciones que corresponden al servidor.

El servidor debería mantener la responsabilidad sobre operaciones que requieren acceso a los datos o seguridad, entre ellas:

- Autenticación de usuarios.
- Autorización y comprobación de permisos.
- Consulta y almacenamiento de cursos.
- Registro de las inscripciones.
- Almacenamiento de información del alumnado.
- Validaciones relacionadas con reglas de negocio.
- Protección de datos y operaciones sensibles.

Por tanto, una validación realizada en el navegador no debería considerarse suficiente para garantizar la seguridad. El Front-End puede mejorar la experiencia del usuario, pero el servidor debe volver a comprobar las condiciones relevantes antes de aceptar una operación.

7. ¿Por qué elegir React para la academia?

La primera razón para elegir React sería su modelo basado en componentes. La aplicación propuesta contiene diferentes partes que pueden aislarse claramente, como los cursos, los filtros, la inscripción y el área privada. La reutilización de componentes puede facilitar tanto el desarrollo como el mantenimiento.

La segunda razón sería su flexibilidad, puesto que el servidor y la API ya existen, el objetivo principal del equipo Front-End es construir una interfaz que consuma esos servicios. React permite centrarse en la capa de interfaz y seleccionar las herramientas adicionales que sean necesarias para el proyecto.

Sin embargo, existe un riesgo importante: la falta de una arquitectura común. Si el equipo no establece previamente criterios sobre componentes, gestión del estado, estructura de carpetas y utilización de dependencias, la flexibilidad puede terminar generando inconsistencias.

8. Comparación con Angular

React y Angular comparten el objetivo general de facilitar el desarrollo de aplicaciones web modernas y utilizan componentes como parte fundamental de la organización de la interfaz. Sin embargo, existen diferencias relevantes.

La primera diferencia es su alcance. React se define como una biblioteca de JavaScript para interfaces de usuario ([React, s. f.-a](#)), mientras que Angular se define como un framework web que incorpora un conjunto amplio de herramientas, APIs y bibliotecas.

La segunda diferencia es el grado de decisión arquitectónica que se deja al equipo. En React es habitual complementar la biblioteca con otras herramientas según las necesidades del proyecto. Angular proporciona una plataforma más integrada y estructurada. Esto puede resultar beneficioso para equipos que prefieren unas convenciones más definidas desde el comienzo.

Además, Angular utiliza TypeScript como una parte importante de su entorno de desarrollo. Su documentación oficial indica que, antes de comenzar, resulta conveniente conocer HTML,

CSS y TypeScript, además de conceptos como clases, fundamentos de TypeScript y decoradores ([Angular, s. f.-b](#)).

Por tanto, la elección dependerá también de las características y experiencia del equipo. React puede ser una opción atractiva cuando se valora la flexibilidad y la construcción de la interfaz mediante componentes, mientras que Angular puede resultar especialmente interesante cuando se busca un framework más integrado y estructurado.

9. Proyecto en el que elegí React y proyecto en el que no sería la primera opción

Elegí React para una plataforma educativa, como la academia planteada en esta actividad. El proyecto contiene diferentes interfaces interactivas y componentes reutilizables, pero el servidor y la API ya están desarrollados. La capacidad de construir la interfaz mediante componentes encaja bien con estas necesidades.

También podría ser apropiado para aplicaciones con diferentes vistas, formularios y elementos interactivos que necesiten actualizarse según las acciones del usuario.

En cambio, para un proyecto extremadamente sencillo, como una web informativa estática de pocas páginas y sin apenas interacción, React no sería necesariamente nuestra primera opción. En ese contexto, incorporar una biblioteca destinada a gestionar interfaces dinámicas puede introducir una complejidad que el proyecto realmente no necesita.

La elección de una tecnología debe realizarse, por tanto, teniendo en cuenta el problema que se pretende resolver y no únicamente las características de la herramienta.

10. Conclusión

Tras analizar React, consideramos que es una opción adecuada para el desarrollo de la interfaz de la academia propuesta. Su principal fortaleza es el modelo basado en componentes, que permite dividir la aplicación en unidades reutilizables y facilita la organización de interfaces complejas. Además, su orientación específica hacia la interfaz permite integrarlo con una API y un servidor ya existentes.

No obstante, su flexibilidad también constituye una limitación. Al no proporcionar por sí mismo todas las decisiones arquitectónicas de un framework completo, el equipo debe establecer criterios y seleccionar herramientas adicionales de forma coherente.

Por ello, nuestra elección sería React para la aplicación de la academia, especialmente si el equipo posee conocimientos de JavaScript y quiere mantener flexibilidad en la construcción del Front-End. No lo elegí necesariamente para una web estática muy sencilla, donde una solución más básica podría ser suficiente.

Bibliografía

Angular. (s. f.). What is Angular? Angular. Recuperado el 7 de octubre de 2026, de <https://angular.dev/docs>

Angular. (s. f.). Essentials. Angular. Recuperado el 7 de octubre de 2026, de <https://angular.dev/essentials>

React. (s. f.-a). Describing the UI. React. Recuperado el 7 de octubre de 2026, de <https://react.dev/learn/describing-the-ui>

React. (s. f.-b). Acknowledgements. React. Recuperado el 7 de octubre de 2026, de <https://react.dev/community/acknowledgements>

React. (s. f.-c). Thinking in React. React. Recuperado el 7 de octubre de 2026, de <https://react.dev/learn/thinking-in-react>

React. (2026). React. GitHub. Recuperado el 7 de octubre de 2026, de <https://github.com/react/react>